KEY — Biweekly Assessment BW05 — Weeks 9–10 (Trainer only)

Do not share with students. Excluded from student pack (`*_KEY.md`).

A. MCQ answers

1. **a** — After `try` / `catch`, whether you succeeded, caught, or `return`ed
2. **a** — Auto-closes the `Scanner` (`AutoCloseable`)
3. **a** — `extends Exception` or `extends RuntimeException`, and you `throw new ...`
4. **a** — Creates or replaces the file (old content is gone)
5. **a** — Stages then records a snapshot
6. **a** — A remote (e.g. GitHub)
7. **a** — `*.class`, `target/`, `.env` (secrets stay out of Git)
8. **a** — DDL — defines structure; PRIMARY KEY uniquely identifies a row
9. **a** — Deletes every row — always use `WHERE`; test with `SELECT` first
10. **a** — DQL — filter, sort, and cap rows (does not change data)
11. **a** — Combine related tables (a foreign key matches a primary key)
12. **a** — Concatenating quotes into SQL lets attackers inject commands (SQL injection)

B. Coding — model points

1. Custom checked `InsufficientFundsException`, `throw` on bad amount / overspend, `try` / `catch` in `main`. After `pay(40)` balance is `60`, so `pay(80)` is the domain failure. `pay(-5)` is `IllegalArgumentException`. No files. No JDBC.

```
public class CanteenPay {
    private int balance;

    CanteenPay(int opening) {
        this.balance = opening;
    }

    void pay(int amount) throws InsufficientFundsException {
        if (amount <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
        if (amount > balance) {
            throw new InsufficientFundsException(
                    "Requested ₹" + amount + ", available ₹" + balance);
        }
        balance -= amount;
        System.out.println("Paid ₹" + amount + ", left ₹" + balance);
    }

    public static void main(String[] args) {
        CanteenPay wallet = new CanteenPay(100);
        try {
            wallet.pay(40);
        } catch (InsufficientFundsException e) {
```

```
            System.out.println("funds: " + e.getMessage());
        }
        try {
            wallet.pay(80);
        } catch (InsufficientFundsException e) {
            System.out.println("funds: " + e.getMessage());
        }
        try {
            wallet.pay(-5);
        } catch (IllegalArgumentException e) {
            System.out.println("input: " + e.getMessage());
        } catch (InsufficientFundsException e) {
            System.out.println("funds: " + e.getMessage());
        }
    }
}

class InsufficientFundsException extends Exception {
    InsufficientFundsException(String message) {
        super(message);
    }
}
```

Expected prints include `Paid ₹40, left ₹60`, then a funds message for `80`, then an input message for `-5`. Award marks for `throw` + custom type + separate catches. Unchecked custom (`extends RuntimeException`) is acceptable if they still catch it.

2. `Path` + UTF-8, header once, `CREATE` + `APPEND` (not a second no-option `writeString`). `throw` on `amount <= 0`. Catch `IOException` around disk work. No JDBC.

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class CanteenCsv {
    static final Path FILE = Path.of("canteen_expenses.csv");

    static void ensureHeader() throws IOException {
        if (Files.notExists(FILE)) {
            Files.writeString(FILE, "item,amount\n", StandardCharsets.UTF_8);
        }
    }

    static void appendRow(String item, int amount) throws IOException {
        if (amount <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
        Files.writeString(FILE, item + "," + amount + "\n", StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }

    public static void main(String[] args) {
        try {
            ensureHeader();
            appendRow("Tea", 45);
            appendRow("Bus", 30);
            try {
                appendRow("Samosa", 0);
            } catch (IllegalArgumentException e) {
```

```
                System.out.println("input: " + e.getMessage());
            }
            System.out.print(Files.readString(FILE, StandardCharsets.UTF_8));
        } catch (IOException e) {
            System.out.println("Could not use CSV: " + e.getMessage());
        }
    }
}
```

Expected on a **missing** file: `input: amount must be positive`, then file text `item,amount` / `Tea,45` / `Bus,30`. Award marks if they used `CREATE` + `APPEND` (or try-with-resources `BufferedWriter` with the same options) and UTF-8. A second no-option `writeString` that wipes the header is a miss.

C. Short answer — model

Boxes first: Member, Book, Loan (or **TechMart** / CampusCart: Customer, Product, Order). A member has many loans (1:N); a book has many loans. Draw that, then `CREATE TABLE`. PRIMARY KEY uniquely identifies one row so a loan can point at **one** member id and **one** book id. One Excel sheet mixes people and copies with no types or uniqueness — anyone can type text in a “due date” cell. Do not require JOIN SQL; naming the relationship is enough. `README.md` / Git are not the point of this item.

Suggested scoring

Section	Marks
A MCQ (1 each)	12
B Coding (6 each)	12
C Short	6
Total	**30**